

Fecha: 06/07/2026	Duración: 2 h	Página 1 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

Ejercicio 1:

Dada la siguiente fórmula de actualización de gradientes:

$$\begin{aligned}
 (1) \quad & G \leftarrow \gamma G + R_{t+1} \\
 (2) \quad & \delta \leftarrow G - \hat{v}(S_t, \mathbf{w}) \\
 (3) \quad & \mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \nabla \hat{v}(S_t, \mathbf{w}) \\
 (4) \quad & \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \delta \nabla \ln \pi(A_t | S_t, \boldsymbol{\theta})
 \end{aligned}$$

1. ¿A qué algoritmo visto en el curso corresponde?
2. Indique que es lo que se está derivando y lo que no. ¿Por qué se tratan así los gradientes?
3. ¿Es esta técnica on u off policy? Justifique.

SOLUCIÓN:

1. ¿Qué algoritmo es?

Es **REINFORCE con baseline** (Monte Carlo Policy Gradient con línea base). Las pistas: el retorno se acumula como $G \leftarrow \gamma G + R_{t+1}$ (cálculo del retorno completo del episodio, recorrido hacia atrás), el error es $\delta = G - \hat{v}(S_t, \mathbf{w})$ (retorno completo menos baseline, no un target bootstrapped), y hay dos conjuntos de parámetros: $\mathbf{w}$ para la función de valor (baseline) y $\boldsymbol{\theta}$ para la política. No es Actor-Critic porque δ usa el retorno completo G y no $R + \gamma \hat{v}(S')$.

2. ¿Qué se deriva y qué no?

- **Se deriva:** $\nabla \hat{v}(S_t, \mathbf{w})$ respecto de $\mathbf{w}$ (parámetros del estimador de valor) y $\nabla \ln \pi(A_t | S_t, \boldsymbol{\theta})$ respecto de $\boldsymbol{\theta}$ (parámetros de la política).
- **No se deriva:** el retorno G (y por lo tanto δ). G es una muestra observada del entorno: una suma de recompensas reales que se trata como una **constante** (el "target").

¿Por qué? Porque el objetivo es ajustar $\hat{v}$ hacia el target G , no ajustar el target hacia $\hat{v}$. Si se derivara también G (o el target en general), no se estaría siguiendo el gradiente correcto del objetivo: δ actúa como un escalar que pondera cuánto y en qué dirección mover los parámetros. En el caso de la política, el teorema del gradiente de políticas dice justamente que el gradiente del desempeño es proporcional a $E[G \cdot \nabla \ln \pi]$, donde G entra multiplicando, sin derivarse.

3. ¿On u off policy?

Es **on-policy**. Los episodios que generan las muestras (S_t, A_t, R_{t+1}) deben ser generados por la propia política $\pi_{\boldsymbol{\theta}}$ que se está optimizando, porque el estimador del gradiente $\delta \nabla \ln \pi(A_t | S_t, \boldsymbol{\theta})$ solo es insesgado cuando las acciones se muestrean de π . No hay corrección por importance sampling

Fecha: 06/07/2026	Duración: 2 h	Página 2 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

ni operador max que permita aprender de otra política de comportamiento. Además, tras cada actualización de θ la política cambia y los datos viejos dejan de ser válidos.

Ejercicio 2:

1. Explicar por qué el bootstrapping de n pasos se considera una generalización de los métodos MC y TD.
2. ¿Qué diferencia hay entre el target de método montecarlo y el de bootstrapping de 2 pasos (tipo off-policy) para estimar Q ? Planteelos.

SOLUCIÓN:

1. n-step bootstrapping como generalización de MC y TD

El target de n pasos usa **n recompensas reales muestreadas** y luego "bootstrapea" con la estimación de valor en el estado alcanzado:

$$G_{t:t+n} = \underbrace{R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{n-1} R_{t+n}}_{n \text{ recompensas reales}} + \underbrace{\gamma^n V(S_{t+n})}_{\text{bootstrap}}$$

- **Con $n = 1$** se recupera exactamente **TD(0)**:
 $G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$
 Una sola recompensa real y bootstrap inmediato.
- **Con $n \rightarrow \infty$** (o $n \geq$ largo del episodio) se llega al estado terminal, donde $V(S_T) = 0$, así que el término de bootstrap desaparece y queda el retorno completo: exactamente **Monte Carlo**.

Es decir, TD(0) y MC son los dos **casos extremos** de una misma familia de targets, y n interpola de forma continua entre ellos. El parámetro n controla el trade-off:

	Más bootstrap (n chico)	Menos bootstrap (n grande)
Sesgo	Mayor (usa estimaciones imperfectas)	Menor
Varianza	Menor	Mayor (acumula muchas muestras aleatorias)
Latencia de actualización	Inmediata	Hay que esperar n pasos (o el fin del episodio)

Fecha: 06/07/2026	Duración: 2 h	Página 3 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

2. Targets para estimar Q

Monte Carlo — retorno completo observado hasta el final del episodio, sin bootstrap:

$$G_t = R_{\{t+1\}} + \gamma \cdot R_{\{t+2\}} + \gamma^2 \cdot R_{\{t+3\}} + \dots + \gamma^{(T-t-1)} \cdot R_T$$

y la actualización es:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [G_t - Q(S_t, A_t)]$$

Bootstrapping de 2 pasos, tipo off-policy (estilo Q-learning) — dos recompensas reales y luego bootstrap con el **máximo** sobre acciones, es decir evaluando la política objetivo greedy en vez de la de comportamiento:

$$G_{\{t:t+2\}} = R_{\{t+1\}} + \gamma \cdot R_{\{t+2\}} + \gamma^2 \cdot \max_a Q(S_{\{t+2\}}, a)$$

y la actualización:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [G_{\{t:t+2\}} - Q(S_t, A_t)]$$

Diferencias clave:

	Monte Carlo	2-step off-policy
Usa estimaciones	No	Sí (bootstrap en t+2)
Sesgo / varianza	Insesgado, alta varianza	Sesgado, menor varianza
Cuándo actualiza	Al terminar el episodio	2 pasos después
Política	Evalúa la que generó los datos	El max evalúa la greedy, ≠ comportamiento

En resumen: MC arrastra toda la aleatoriedad de la trayectoria restante, mientras que el target de 2 pasos **corta** la trayectoria en t+2 y reemplaza el resto del retorno por la estimación actual de Q evaluada de forma greedy. Eso mete sesgo pero baja la varianza, permite actualizar sin esperar al fin del episodio, y al usar max en lugar de la acción realmente tomada, aprende off-policy.

Fecha: 06/07/2026	Duración: 2 h	Página 4 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

Ejercicio 3:

Supongamos que estás diseñando un estimador de la función valor $Q(s)$ para un agente de DQN que opera en un entorno con un espacio de observación de dimensión 128 (por ejemplo, un robot que percibe su entorno mediante 128 sensores). El agente puede: avanzar, frenar, o hacer noop (no hacer nada).

Proponés utilizar una red neuronal con la siguiente arquitectura:

Capa de Entrada: 128 neuronas

Capa Oculta 1: 64 neuronas

Capa Oculta 2: 64 neuronas

Capa de Salida: ? neuronas

Se utilizan funciones de activación ReLU para todas las capas.

Capa	# de Neuronas de Entrada	# de Neuronas de Salida	# de Pesos	# de Sesgos	# Total de Parámetros
FC1					
FC2					
FC3					

Se pide:

1. ¿Cuántas neuronas tiene la capa de salida?
2. Completa la tabla indicando el número de pesos, sesgos y parámetros totales en cada capa.
3. Calcula el número total de parámetros entrenables de la red.

Fecha: 06/07/2026	Duración: 2 h	Página 5 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

SOLUCIÓN:

1. Neuronas de salida: en DQN la red estima $Q(s, a)$ para todas las acciones a la vez, con una salida por acción. Hay 3 acciones (avanzar, frenar, noop) → **3 neuronas**.

2. Tabla:

Capa	# de Neuronas de Entrada	# de Neuronas de Salida	# de Pesos	# de Sesgos	# Total de Parámetros
FC1	128	64	$128 \times 64 = 8.192$	64	8.256
FC2	64	64	$64 \times 64 = 4.096$	64	4.160
FC3	64	3	$64 \times 3 = 192$	3	195

(Regla: pesos = entradas $\times$ salidas; sesgos = uno por neurona de salida.)

3. Total de parámetros entrenables:

$$8.256 + 4.160 + 195 = 12.611$$

Fecha: 06/07/2026	Duración: 2 h	Página 6 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

Ejercicio 4:

Dados el siguiente pseudo-código:

Prioritized sweeping for a deterministic environment

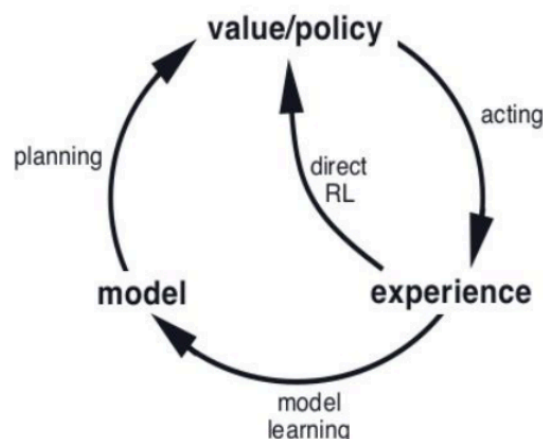
Initialize $Q(s, a)$, $Model(s, a)$, for all s, a , and $PQueue$ to empty

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow policy(S, Q)$
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Model(S, A) \leftarrow R, S'$
- (e) $P \leftarrow |R + \gamma \max_a Q(S', a) - Q(S, A)|$.
- (f) if $P > \theta$, then insert S, A into $PQueue$ with priority P
- (g) Loop repeat n times, while $PQueue$ is not empty:
 - $S, A \leftarrow first(PQueue)$
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 - Loop for all $\bar{S}, \bar{A}$ predicted to lead to S :
 - $\bar{R} \leftarrow$ predicted reward for $\bar{S}, \bar{A}, S$
 - $P \leftarrow |\bar{R} + \gamma \max_a Q(S, a) - Q(\bar{S}, \bar{A})|$.
 - if $P > \theta$ then insert $\bar{S}, \bar{A}$ into $PQueue$ with priority P

Se pide:

- Explicar en detalle para el algoritmo A) qué instrucciones realizan cada uno de los pasos "acting", "direct RL", "model learning" y "planning", según el siguiente diagrama de planificación online:



- Explique por qué el barrido priorizado es más eficiente que la versión estándar de DynaQ

Fecha: 06/07/2026	Duración: 2 h	Página 7 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

SOLUCIÓN:

1. Mapeo del pseudo-código al diagrama de planificación online:

- **Acting** (política → experiencia): pasos **(a)**, **(b)**, **(c)** — se toma el estado actual S , se elige la acción A con la política derivada de Q , se ejecuta A en el entorno real y se observan R y S' . Aquí se genera la experiencia real.
- **Model learning** (experiencia → modelo): paso **(d)** — $\text{Model}(S, A) \leftarrow R, S'$. Se guarda la transición observada en el modelo (determinístico, por eso alcanza con almacenar el último R, S').
- **Direct RL** (experiencia → valor/política): pasos **(e)**, **(f)** — se calcula el error TD de la experiencia real $P = |R + \gamma \max_a Q(S', a) - Q(S, A)|$ y, si es significativo ($P > \theta$), se encola. En esta variante la experiencia real no actualiza Q inmediatamente, sino que su efecto sobre los valores queda "priorizado" y se materializa vía la cola; es la forma en que la experiencia directa influye en value/policy.
- **Planning** (modelo → valor/política): paso **(g)** — el loop de n repeticiones: se extrae el par de mayor prioridad de la $P\text{Queue}$, se simula la transición con el **modelo** ($R, S' \leftarrow \text{Model}(S, A)$), se actualiza Q con la regla de Q -learning, y luego se propaga hacia atrás: se recalculan las prioridades de todos los predecesores (S^-, A^-) ($\bar{S}, \bar{A}$) (S^-, A^-) que llevan a S y se encolan si superan el umbral.

2. ¿Por qué el barrido priorizado es más eficiente que Dyna-Q estándar?

En Dyna-Q estándar, los n pasos de planning eligen pares (S, A) **uniformemente al azar** entre los ya visitados. La gran mayoría de esas actualizaciones son inútiles: el target casi no cambió, así que $Q(S, A)$ queda prácticamente igual y se desperdicia cómputo.

El barrido priorizado concentra el cómputo donde importa: solo actualiza pares cuyo error TD estimado es grande ($P > \theta$), empezando por el mayor, y cuando un valor $Q(S)$ cambia significativamente, **propaga ese cambio hacia atrás** a sus estados predecesores, que ahora sí tienen targets desactualizados. Así los cambios de valor (por ejemplo, al descubrir la meta por primera vez) se difunden rápidamente por el espacio de estados en orden de relevancia, en lugar de esperar a que el muestreo uniforme "acierte" por casualidad los estados afectados. El resultado típico es alcanzar la política óptima con órdenes de magnitud menos actualizaciones.

Fecha: 06/07/2026	Duración: 2 h	Página 8 de 8
Evaluación: Parcial	Uso de Calculadora: SI	
Materia: Agentes Inteligentes	Uso de Material: NO	
Turno: Nocturno	Puntaje Máximo: 50 Puntos	

Fórmulas de interés:

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')]$$

$$q_{\pi}(s,a) = \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')]$$